Assignment

2110413 COMPUTER SECURITY

Assignment 5
Public Key Infrastructure

Computer Engineering, Chulalongkorn University

Worralop Srichainont

Student ID: 6632200221

October 5, 2026

Contents

1 Setup **1**

2 Exercises 1–9: TLS connection and certificates **1**

2.1 Exercise 1: With and without `-CAfile` 1

2.2 Exercise 2: What does the `verify error` mean? 2

2.3 Exercise 3: The server certificate 2

2.4 Exercise 4: Is there an intermediate certificate? 3

2.5 Exercise 5: Is more than one organization involved? 4

2.6 Exercise 6: Role of `ca-certificates.crt` 4

2.7 Exercise 7: How many certificates are in the file? 5

2.8 Exercise 8: A root certificate 5

2.9 Exercise 9: DER to PEM 6

3 Exercise 10: Certificate validation in Python **6**

4 Exercise 11: Class 1 and Class 3 certificates **7**

5 Exercise 12: A compromised root CA **8**

5.1 12a: Further attacks 8

5.2 12b: Can we rely on CRLs or OCSP? 8

1 Setup

Commands were run on Windows 11 with OpenSSL 3.5.9 (Miniconda), Python 3 with PyOpenSSL 26.4.0 and `pem`, and MiKTeX `xelatex`. The trusted root bundle `ca-certificates.crt` is the Mozilla CA list from `curl.se/ca/cacert.pem`. Scripts and raw outputs are in `code/pki/` and `code/pki/out/`. This OpenSSL has no default trust store, so the effect of `-CAfile` is visible directly. The observed certificates are the ones served on the day of the test and may change later.

2 Exercises 1–9: TLS connection and certificates

2.1 Exercise 1: With and without `-CAfile`

Commands

```
openssl s_client -connect twitter.com:443
openssl s_client -connect twitter.com:443 -CAfile ca-certificates.crt
openssl s_client -connect twitter.com:443 -CApath empty_dir
```

Typing `GET / HTTP/1.0` and Enter twice returned `HTTP/1.1 400 Bad Request` in all three runs. This is normal, since no valid host or resource was requested.

Command	Verify return code	Result
no option	20 (unable to get local issuer certificate)	`verify error`
`-CAfile ca-certificates.crt`	0 (ok)	`Verification: OK`
`-CApath empty_dir`	20 (unable to get local issuer certificate)	`verify error`

Table 1: Result of the three `s_client` runs.

Difference. Both commands open the same encrypted TLS session (TLS 1.3, `TLS_AES_256_GCM_SHA384`) and receive the same certificate chain. The only difference is whether the client can *authenticate* the server. Without `-CAfile` the client has no trusted root, so it cannot validate the chain and prints a `verify error`. With `-CAfile` it finds a trusted root and prints `Verification: OK`. The second run also shows `depth=3` (ISRG Root X1) in its verify lines.

2.2 Exercise 2: What does the `verify error` mean?

First command (no trust store)

```
depth=2 C=US, O=ISRG, CN=Root YR
verify error:num=20:unable to get local issuer certificate
...
Verification error: unable to get local issuer certificate
```

Error 20 means OpenSSL could not build a path from the server certificate up to a certificate it trusts. The last certificate in the chain was `Root YR`, and its issuer (ISRG Root X1) is not in any trust store the client knows about. The connection is still encrypted, but the client has no proof that it is talking to the real twitter.com. A man in the middle could present a self-made chain and the client could not tell the difference. This is why the error must not be ignored in practice.

2.3 Exercise 3: The server certificate

The server certificate was saved as `twitter_com.cert` (the first `BEGIN/END CERTIFICATE` block) and displayed with `openssl x509 -in twitter_com.cert -text`. The important fields are:

openssl x509 -in twitter_com.cert -text (excerpt)

```
Version: 3 (0x2)
Serial Number: 05:1c:d1:ae:fb:de:89:df:b5:9b:40:37:86:aa:c2:cf:0a:2f
Signature Algorithm: sha256WithRSAEncryption
Issuer: C=US, O=Let's Encrypt, CN=YR1
Validity
    Not Before: Sep 28 19:44:38 2026 GMT
    Not After : Dec 27 19:44:37 2026 GMT
Subject: CN=*.twitter.com
Subject Public Key Info: rsaEncryption, 2048 bit, Exponent 65537
X509v3 Key Usage: critical  Digital Signature, Key Encipherment
X509v3 Extended Key Usage:  TLS Web Server Authentication
X509v3 Basic Constraints: critical  CA:FALSE
X509v3 Subject Key Identifier:   82:2A:D6:A3:...:1C:D3:4A
X509v3 Authority Key Identifier: 1F:2F:35:BE:...:68:FB
Authority Information Access: CA Issuers - URI:http://yr1.i.lencr.org/
X509v3 Subject Alternative Name:
    DNS:*.twitter.com, DNS:cdn.syndication.twitter.com, DNS:twitter.com
X509v3 Certificate Policies: Policy: 2.23.140.1.2.1
X509v3 CRL Distribution Points: URI:http://yr1.c.lencr.org/48.crl
CT Precertificate SCTs: (2 timestamps)
Signature Algorithm: sha256WithRSAEncryption  Signature Value: ...
```

Field	Meaning
Version	X.509 version (v3 allows extensions).
Serial Number	Unique number given by the issuer; used in revocation lists.
Signature Algorithm	Algorithm the issuer used to sign this certificate.
Issuer	The CA that signed the certificate (Let's Encrypt YR1).
Validity	Time window in which the certificate may be trusted (90 days here).
Subject	The identity being certified (`*.twitter.com`).
Subject Public Key Info	The subject's public key and its algorithm (RSA-2048).
Key Usage / Extended Key Usage	What the key may do: sign and encrypt keys, TLS server authentication.
Basic Constraints	`CA:FALSE`: this certificate cannot issue other certificates.
Subject/ Authority Key Identifier	Hash of this key and of the issuer's key; helps find the issuer.
Authority Information Access	Where to download the issuer's certificate.
Subject Alternative Name	Hostnames the certificate is valid for. Modern clients check this, not the CN.
Certificate Policies	Policy OIDs; `2.23.140.1.2.1` is a domain-validated certificate.
CRL Distribution Points	Where to fetch the revocation list.
SCTs	Proof the certificate was logged in Certificate Transparency logs.
Signature Value	The issuer's digital signature over all fields above.

Table 2: Fields of an X.509 certificate.

2.4 Exercise 4: Is there an intermediate certificate?

Yes. The issuer of the server certificate is `CN=YR1, O=Let's Encrypt`, which is not a root. The AIA field gives its download location, so I downloaded it:

Intermediate certificate

```
curl -o intermediate.cert http://yr1.i.lencr.org/
openssl x509 -inform der -in intermediate.cert -text
```

Excerpt

```
Issuer: C=US, O=ISRG, CN=Root YR
Validity: Sep  3 00:00:00 2025 GMT  to  Sep  2 23:59:59 2028 GMT
Subject: C=US, O=Let's Encrypt, CN=YR1
X509v3 Key Usage: critical  Digital Signature, Certificate Sign, CRL Sign
X509v3 Basic Constraints: critical  CA:TRUE, pathlen:0
Authority Information Access: CA Issuers - URI:http://yr.i.lencr.org/
```

It is an intermediate CA (`CA:TRUE`, `pathlen:0`). Its purpose is to sign end-entity certificates on behalf of the root. The root private key can then stay offline and rarely

used. If the intermediate is compromised, only that intermediate must be revoked, not the root. `pathlen:0` means it can sign leaf certificates but not further CAs. The file's SHA-256 fingerprint equals that of the second certificate sent by the server, so the server was already providing it.

2.5 Exercise 5: Is more than one organization involved?

Yes, there are separate certificate subjects. The chain presented by the server is:

Chain (from s_client -showcerts)

```
0 s:CN=*.twitter.com            i:O=Let's Encrypt, CN=YR1
1 s:O=Let's Encrypt, CN=YR1     i:O=ISRG, CN=Root YR
2 s:O=ISRG, CN=Root YR          i:O=Internet Security Research Group, CN=ISRG
  ↪  Root X1
(in the root store)
3 s:O=Internet Security Research Group, CN=ISRG Root X1   (self-signed)
```

The server certificate is signed by an intermediate named Let's Encrypt (the CA that serves the public), which is signed by Root YR, which is in turn signed by the self-signed root ISRG Root X1 operated by the Internet Security Research Group. These are different certificate subjects with different `O` values. Let's Encrypt is the service run by ISRG, so it is one organization family, but there are two levels of intermediates between the server and the root. Twitter itself is only the subject, not a CA. The reason for this design: the root key is kept offline for safety, and day-to-day signing is done by intermediates.

2.6 Exercise 6: Role of `ca-certificates.crt`

It is the *trust store* (root store): a file of root CA certificates that the client trusts. During verification, OpenSSL builds a chain from the server certificate to a certificate in this file and checks every signature, validity period, and constraint along the way. A chain that ends at a certificate in this file is accepted. Everything else is rejected. Trust in all websites is therefore reduced to trust in the roots listed here.

2.7 Exercise 7: How many certificates are in the file?

Counting

```
grep -c "BEGIN CERTIFICATE" ca-certificates.crt
openssl storeutl -noout -text -certs ca-certificates.crt | grep "Total found"
```

Output

```
121
Total found: 121
```

The Mozilla bundle I downloaded contains **121** root certificates. The count depends on the bundle version, so the course file may give a different number.

2.8 Exercise 8: A root certificate

I extracted `ISRG Root X1` (the root at the top of twitter's chain) from the bundle with a small Python script that splits the PEM blocks and checks each subject, and displayed it with `openssl x509 -in root_isrg_x1.pem -text`.

	Twitter certificate	Intermediate (YR1)	Root (ISRG Root X1)
Subject	CN=*.twitter.com	O=Let's Encrypt, CN=YR1	O=Internet Security Research Group, CN=ISRG Root X1
Issuer	O=Let's Encrypt, CN=YR1	O=ISRG, CN=Root YR	**same as its subject**
Valid until	Dec 27 2026	Sep 2 2028	Jun 4 2035
Basic Constraints	CA:FALSE	CA:TRUE, pathlen:0	CA:TRUE
Key usage	Digital Signature, Key Encipherment	Digital Signature, Cert Sign, CRL Sign	Cert Sign, CRL Sign

Table 3: Comparison of the three certificates.

Yes, the root has an Issuer field, and it is identical to its Subject. It is *self-signed*: the root vouches for itself, so nobody above it can vouch for it. Its trust comes only from being included in the root store. The leaf's issuer equals the subject of the intermediate, and the intermediate's issuer names the next certificate up, which gives the chain of trust. The root has the longest validity (to 2035), the intermediate a few years, and the leaf only 90 days, since keys that are used more often are replaced sooner.

2.9 Exercise 9: DER to PEM

The intermediate downloaded from the AIA URL was in DER format (binary), so it was converted to PEM:

Conversion

```
openssl x509 -inform der -in intermediate.cert -out intermediate.pem
```

3 Exercise 10: Certificate validation in Python

I used the supplied code (PyOpenSSL `X509Store` and `X509StoreContext`), modified to take the host name, fetch the chain from the server with `openssl s_client -showcerts`, and treat the first certificate as the target and the others as intermediates. Each host was tested in two ways: with the root store only, and with the root store plus the intermediates.

verify_chain.py (core)

```
def verify_chain_of_trust(cert_pem, trusted_cert_pems):
    certificate = crypto.load_certificate(crypto.FILETYPE_PEM, cert_pem)
    store = crypto.X509Store()
    for trusted_cert_pem in trusted_cert_pems:
        trusted_cert = crypto.load_certificate(crypto.FILETYPE_PEM,
        ↪  trusted_cert_pem)
        store.add_cert(trusted_cert)
    store_ctx = crypto.X509StoreContext(store, certificate)
    try:
        store_ctx.verify_certificate()   # None means validated
        return True, "ok"
    except crypto.X509StoreContextError as e:
        return False, str(e)

def verify(host, use_intermediate=True):
    chain = fetch_chain(host)               # s_client -showcerts
    cert, int_certs = chain[0], chain[1:]
    trusted = [str(p) for p in pem.parse_file("./ca-certificates.crt")]
    if use_intermediate:
        trusted.extend(int_certs)
    return verify_chain_of_trust(cert, trusted)
```

Output of python verify_chain.py

```
twitter.com       [root store only ] -> FAILED (unable to get local issuer
  ↪ certificate)
twitter.com       [with intermediate] -> Certificate verified
google.com        [root store only ] -> FAILED (unable to get local issuer
  ↪ certificate)
google.com        [with intermediate] -> Certificate verified
www.chula.ac.th   [root store only ] -> FAILED (unable to get local issuer
  ↪ certificate)
www.chula.ac.th   [with intermediate] -> Certificate verified
classdeedee.cloud.cp.eng.chula.ac.th [root store only ] -> FAILED (unable to
  ↪ get local issuer certificate)
classdeedee.cloud.cp.eng.chula.ac.th [with intermediate] -> Certificate
  ↪ verified
```

Host	Chain sent by server (leaf issuer → …)	Trust anchor in bundle
twitter.com	Let's Encrypt YR1 → ISRG Root YR	ISRG Root X1
google.com	Google Trust Services WR2 → GTS Root R1	GlobalSign Root CA
www.chula.ac.th	Sectigo DV R36 → Sectigo Root R46	USERTrust RSA Certification Authority
classdeedee.cloud.cp.eng.chula.ac.th	Let's Encrypt YR2 → ISRG Root YR	ISRG Root X1

Table 4: Chain of each host and the trust anchor that validates it.

All four certificates were verified when the intermediates were supplied. Without the intermediates, validation fails for every host, because the root store contains only roots and the issuer of the leaf is an intermediate that the store does not know. This is why servers must send the intermediate certificates along with their own. Note that the classdeedee server returns the certificate `CN=cloud.cp.eng.chula.ac.th`, a certificate of the parent domain. The script checks the chain only: it does not check that the name matches the host or that the certificate is not revoked.

4 Exercise 11: Class 1 and Class 3 certificates

The *class* describes how strongly the CA checks the identity of the applicant before it signs (the assurance level of the certificate policy).

Class 1 (low assurance): only a minimal check, usually that the applicant controls an email address. No identity documents are required. Typical uses: personal email signing and encryption (S/MIME), and low-risk logins where the user only needs to be recognised, not identified. It is cheap and quick but cannot prove who the person or organisation really is, so it is unsuitable for banking or commerce.

Class 3 (high assurance): the CA verifies the real identity of the person or legal entity with documents, often in person or with a notary, and checks the applicant's authority. Typical uses: certificates for commercial web servers, code signing, and certificates for organisations and CAs. It is more expensive and slower but allows relying parties to trust that the named organisation is real.

A Class 1 certificate has uses that Class 3 does not need: free personal email protection where convenience matters and identity does not. A Class 3 certificate is needed where a wrong identity causes damage (e-commerce, signed software), and a Class 1 certificate would not be accepted by relying parties there. Today these classes have largely been replaced by validation types (DV, OV, EV), which follow the same idea.

5 Exercise 12: A compromised root CA

5.1 12a: Further attacks

If a root CA in the client's root store is controlled by an attacker, the attacker can sign a certificate for *any* name, and every client that trusts that root will accept it as valid. Possible attacks:

- **Man-in-the-middle on HTTPS:** issue a certificate for `bank.example`, then intercept, read, and modify encrypted traffic with no browser warning.
- **Impersonation and phishing:** fake sites with a valid padlock.
- **Malware and updates:** sign malicious code or updates that seem to come from a trusted vendor, if the root also covers code signing.
- **Forged email, documents, and client certificates** (S/MIME, document signing, certificate-based login).
- **Targeted or mass interception** (for example by a hostile network), which can go unnoticed for months because the forged certificates look valid.

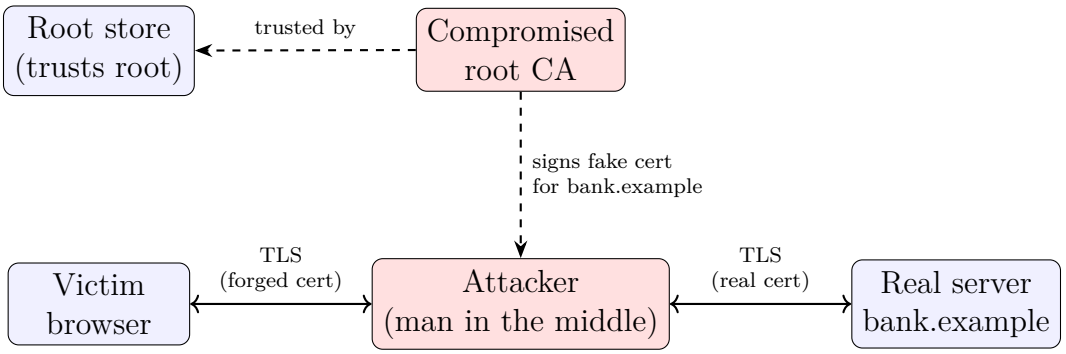


Figure 1: Attack setup: the attacker uses a certificate signed by the compromised root and relays traffic to the real server.

5.2 12b: Can we rely on CRLs or OCSP?

No, not reliably.

- Revocation information is signed by the CA. If the root key or its infrastructure is controlled by the attacker, the attacker can also sign CRLs and OCSP responses, marking the forged certificates as "good", or simply never publish a revocation.
- A root certificate is self-signed and has no issuer above it that could revoke it. Clients do not check CRL/OCSP for the trust anchor. The only fix is out-of-band:

the browser or OS vendor removes the root from the root store with an update.

- In practice, many clients *soft-fail* (accept the certificate when the OCSP server is unreachable), so an attacker in the middle can simply block OCSP requests. CRLs also have a delay between revocation and distribution.
- Since the compromise is unnoticed for months, nothing would be revoked in the meantime anyway.

Additional defences are needed: removing the root from trust stores, Certificate Transparency monitoring (forged certificates become visible in logs), certificate or key pinning, HSTS, and limiting what each root may issue (name constraints).